CS6493: Natural Language Processing - Assignment 1

Instructions

1. Due at 10 pm, Mar. 11, 2026;
2. You can submit your answers by **a single PDF with the code package** or **a single Jupiter notebook** containing both the answers and the code;
3. For the coding questions, besides the code, you are encouraged to additionally give some descriptions of your code design and its workflow. Detailed analysis of the experimental results is also preferred;
4. Total marks are 100;
5. If you have any questions, please post your questions on the Canvas-Discussion forum or contact TA Mr. Junyi ZHOU (junyizhou8-c@my.cityu.edu.hk).

Question 1

(**11 marks**) We could represent the word with one-hot encoding in a language model instead of ascii representations. That is, we represent the word w with:

$$\overbrace{[0, 0, ..., 1, ..., 0, 0]}^{\|V\|}$$

where V is our vocabulary and $\|V\|$ is the size of it. Consider a vocabulary $\mathcal{V} = \{\texttt{cat}, \texttt{dog}, \texttt{run}, \texttt{runs}\}$.

1. **Representation Construction (2 points)**
   Write the standard one-hot vector representation for each word in $\mathcal{V}$, assuming words are ordered lexicographically.

2. **Inner Product and Similarity (3 points)**
   Compute the dot product between the one-hot vectors of any two distinct words.
   What does this result imply about semantic similarity under one-hot encoding? Why is this problematic for modeling linguistic relationships?

3. **Parameter Efficiency (3 points)**
   Suppose these one-hot vectors are used as input to a linear layer with weight matrix $\mathbf{W} \in \mathbb{R}^{4\times d}$, where $d = 128$.
   How many parameters are required to represent the entire embedding table for $\mathcal{V}$?
   If the vocabulary size grows to $|\mathcal{V}| = 50{,}000$, how many parameters would be needed? List two practical issues this scaling causes.

4. **Failure to Capture Morphology (2 points)**
   The words `run` and `runs` exhibit a clear morphological relationship (verb inflection).
   Can this relationship be automatically recognized from their one-hot representations? Why or why not?
   What challenge does this pose for generalizing to unseen inflected forms (e.g., from `jump` to `jumps`)?

Question 2

(**24 marks**) In the field of NLP, for a sequence of text $S = \{w_1, w_2, ..., w_T\}$, the statistical language model transforms the joint probability of a sequence into a product of conditional probabilities:

$$P(S) = P(w_1, w_2, ..., w_T) = \prod_{t=1}^{T} p(w_t | w_1, w_2, ..., w_{t-1})$$

However, this formulation is not useful in practice due to the huge computational costs. We prefer to use its simplified version - the n-gram model:

$$p(w_t | w_1, w_2, ..., w_{t-1}) \approx p(w_t | w_{t-n+1}, ..., w_{t-1})$$

We usually use bi-gram (n=2) or tri-gram (n=3) to model the language. Consider the sentence:

"I am taking CS6493 this semester and studying NLP is really fascinating"

Assume words are separated by whitespace and punctuation is ignored.

1. **n-gram Context Extraction and Sparsity Analysis (8 points)**

   a. List all **bigrams** where the second word is either `CS6493` or `NLP`.

   b. List all **trigrams** where the third word is either `CS6493` or `NLP`.

2. **Critical Evaluation of n-gram Models (6 points)**

   a. Explain why n-gram models cannot generalize to novel sequences composed of known words.

   b. Discuss how the Markov assumption limits the modeling of long-range dependencies. Provide a concrete example where this causes failure.

   c. Why do n-gram models require large corpora to perform well? Relate your answer to the *curse of dimensionality*.

3. **Neural Language Model Design and Parameter Sensitivity (10 points)**

   The following PyTorch code defines a neural language model:

```python
class LanguageModeler(nn.Module):
    def __init__(self, vocab_size, embedding_dim, context_size, hidden_size=128):
        super(LanguageModeler, self).__init__()
        self.embeddings = nn.Embedding(vocab_size, embedding_dim)
        self.linear1 = nn.Linear(context_size * embedding_dim, hidden_size)
        self.linear2 = nn.Linear(hidden_size, vocab_size)

    def forward(self, inputs):
        embeds = self.embeddings(inputs).view((1, -1))
        out = F.relu(self.linear1(embeds))
        out = self.linear2(out)
        log_probs = F.log_softmax(out, dim=1)
        return log_probs
```

   You aim to train a **four-gram model** (predict next word from previous three) on the given sentence for 10 epochs.

   a. Describe how you would preprocess the sentence to generate training examples of the form $(w_{t-3}, w_{t-2}, w_{t-1}) \rightarrow w_t$.

   b. Write a minimal training loop (pseudocode or Python) that trains the model for one epoch using the generated data.

   c. Compare the expected training dynamics (e.g., loss convergence) when using embedding dimensions of 32, 64, and 128. Discuss trade-offs in capacity, generalization, and computational cost.

Question 3 (45 marks)

Training word embeddings can be achieved through different approaches. Two popular methods are **Word2Vec** and **GloVe** (Global Vectors for Word Representation). While Word2Vec learns word embeddings by predicting context words from a target word (Skip-gram) or predicting a target word from its context (CBOW), GloVe learns embeddings by factorizing the global word co-occurrence matrix.

Word2Vec Skip-gram

Word2Vec Skip-gram learns word embeddings by maximizing the probability of observing context words given a target word. In practice, negative sampling is commonly used to efficiently approximate the softmax objective.

Given a target word w_i and a context word w_o, the negative sampling objective is:

$$\mathcal{L} = \log \sigma \left(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_i} \right) + \sum_{k=1}^{K} \mathbb{E}_{w_k \sim P_n(w)} \left[\log \sigma \left(-\mathbf{u}_{w_k}^\top \mathbf{v}_{w_i} \right) \right],$$

where $\mathbf{v}_w$ denotes the input (target) word embedding, $\mathbf{u}_w$ denotes the output (context) word embedding, K is the number of negative samples, $\sigma(\cdot)$ is the sigmoid function, and $P_n(w)$ is the noise distribution.

GloVe

GloVe minimizes the weighted least squares objective on the global word–word co-occurrence matrix:

$$J = \sum_{i,j=1}^{V} f(X_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2,$$

where X_{ij} is the number of times word j occurs in the context of word i, $f(\cdot)$ is a weighting function that helps prevent very frequent co-occurrences from dominating, and $w_i, \tilde{w}_j$ are word vectors with bias terms $b_i, \tilde{b}_j$.

Tasks

1. Train a **Word2Vec Skip-gram** model with negative sampling on the provided `wiki_corpus.txt`. Use appropriate hyperparameters such as embedding dimension, window size, and negative sampling rate. (15 marks)

2. Train a **GloVe** model on the same `wiki_corpus.txt`. Students may either implement the algorithm from scratch or use existing GloVe implementations or libraries with comparable hyperparameters (same embedding dimension and context window as your Skip-gram model). (15 marks)

3. Compare and evaluate the quality of the two models:

   - Perform qualitative evaluation by examining nearest neighbors for selected words (e.g., "Australia", "YMCA", "South", "building").
   - Investigate the effect of at least one hyperparameter (e.g., embedding dimension: 50, 100, 200, or context window size: 2, 5, 10) on the learned embeddings.

   (15 marks)

Question 4

(**20 marks**) Splitting a text into smaller chunks is a task that is harder than it looks, and there are multiple ways of doing so.

1. Simple space tokenization has limitations when processing certain types of text. From the examples below, select at least **TWO** cases and explain why space tokenization would be problematic for each. **(5 marks)**
   - Compound words (e.g., "state-of-the-art")
   - Contractions (e.g., "don't", "we're")
   - URLs/emails (e.g., "user@email.com")
   - Hashtags (e.g., "#MachineLearning")
   - Numbers with units (e.g., "3.14km")

2. Transformers models use a hybrid approach between word-level and character-level tokenization called **subword** tokenization. BPE(Byte-Pair Encoding) is a subword-level tokenization approach introduced in *Neural Machine Translation of Rare Words with Subword Units (Sennrich et al., 2015).* . BPE relies on a pre-tokenizer that splits the training data into words. Pretokenization can be as simple as space tokenization. Let us assume that after pre-tokenization, the following set of words including their frequency has been determined:

   $$(old, 10), (older, 5), (oldest, 8), (hug, 8), (pug, 4), (hugs, 5)$$

   We obtain an base vocabulary:

   $$o, l, d, e, r, s, t, h, u, g, p$$

   Splitting all words into symbols in the base vocabulary, we obtain:

   $$(o, l, d, 10), (o, l, d, e, r, 5), (o, l, d, e, s, t, 8), (h, u, g, 8), (p, u, g, 4), (h, u, g, s, 5)$$

   BPE then counts the frequency of each possible symbol pair and picks the symbol pair that occurs most frequently. In the above example,"o" followed by "l" is present $10+5+8 = 23$ times. Thus, the first merge rule the tokenizer learns is to group all "o" symbols followed by an "l" symbol together. Next, "ol" is added to the vocabulary. The set of words then becomes:

   $$(ol, d, 10), (ol, d, e, r, 5), (ol, d, e, s, t, 8), (h, u, g, 8), (p, u, g, 4), (h, u, g, s, 5)$$

   This process will run iteratively. The vocabulary size, i.e. the base vocabulary size + the number of merges, is a hyperparameter to choose. The learned merge rules would then be applied to new words (as long as those new words do not include symbols that were not in the base vocabulary). The word not in the base vocabulary would be repersented as "[unk]". Implement this BPE tokenizer, set the vocabulary size as 16 and train this BPE tokenizer to finish the iterative proecss. Use the trained tokenizer to tokenize the words below:(**15 marks**)

   $$\{hold, oldest, older, pug, mug, hugging face\}.$$